DRAFT Proposal: SKL as Single Source of Truth, Publishing into Visier

Oct 29, 2026 · @jeffrey

Recommendation

The AI Strategy & EA Team proposes the SKL as the single source of truth (SSOT) for vocabularies, taxonomies, ontologies, rules, constraints, logic, and calculations, with governed outbound services that publish the parts Visier needs. Visier becomes a consumption-only execution point: nothing is authored or edited there.
Recommendation: Approve the following decisions:

1. Establish the SKL as the SSOT aligned with the architecture principles (per our onsite EA mtg)
2. Set API-first publishing as the standard pattern for downstream execution points
3. Define a pilot scoped to the downturn workforce advisory vocabularies, with Visier as the first target endpoint

Assumptions (not yet validated)

Teams typically curate semantics inside Visier, which is an analytics semantic layer, not a knowledge-management system. Its public documentation shows the limits:

- Visier models subjects, properties, dimensions, concepts, and metrics. Its Dimensions API is Alpha and limited to leveled and parent-child dimensions.
- Its SQL-like query language works within one tenant, and queries across analytic objects use only the attributes those objects share. *Note: I was not able to find public documentation of RDF/OWL import, SPARQL, reasoning, or evidence of federated queries*.
- Its skills ontology (14,000+ skills, 3,300+ occupations, built on ESCO) is a product feature, and customizing it is listed as a future capability (something we should do in a knowledge management system using the KE Workbench with Semantic Governance).

If Visier becomes the master, poly-hierarchy, typed relationships, mappings, and rules have nowhere to live, and the other consumers (Copilot, Compass, HRScale, partner APIs) cannot reuse what is curated there. Two masters will drift.

Proposal

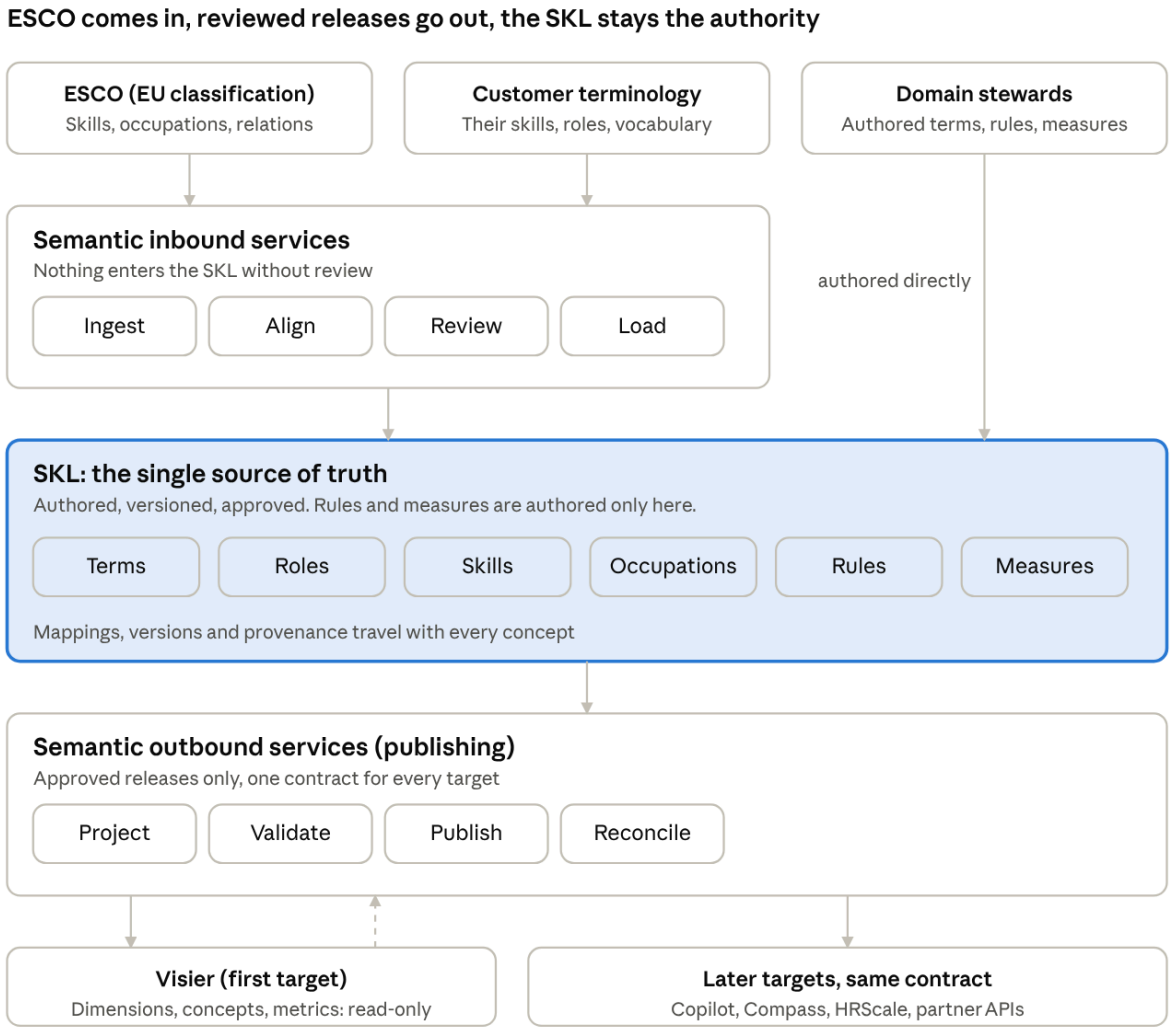
The SKL is the only place where meaning is authored and contextual relevancy mapped. Three layers, with a one-way flow from authority to execution:

1. **SKL (authority):** authoring, governance, versioning, and the canonical store for vocabularies, taxonomies, ontologies, mappings, rules, constraints, logic, and calculations.

2. **Outbound publishing services (API layer):** project SKL content into the shape a target needs, validate it, deliver it, and reconcile what the target holds. One service contract, one adapter per execution point.
3. **Execution points:** Visier first, others later on the same contract. They consume published content and never edit it .Visier reports back only reconciliation status (what it holds, and whether it matches). It never sends definitions back.

End-to-end architecture

Content enters the SKL only through governed inbound services or direct steward authoring, and leaves only through outbound services.



The SKL is the only layer that holds all six element types. Visier sits at the far end and reports reconciliation status back, never definitions.

Fit with architecture principles

Confirm the list against enterprise standards before circulating.

Principle	How the proposal satisfies it
Single source of truth for rules, vocabularies, taxonomies, ontologies	The SKL is the only place meaning is authored. Visier holds projections. *If someone wants to change a term, they change it in the SKL, and then publish it into Visier. If Visier's copy ever differs from the SKL, that's treated as drift (an error to fix), not as a competing opinion.*
Extend platforms, don't replace them	Visier stays the analytics engine. The SKL adds the definitions it cannot hold.
Deterministic core	Eligibility, compliance & calculations run as governed logic in services. Visier receives results and dimensions and does not modify, extend, or reimplement the rules.
Reusable services assembled per tier	Publishing is one service contract with an adapter per target. *Leverage Visier's API to handle the translations (turns SKL taxa into dimension types)*
Define once, publish many	One definition feeds Visier, Copilot, Compass, HRScale and partner APIs.
Separate meaning from execution	Definitions live in the SKL. Execution points (like Visier) apply them.
Consumption only downstream	Published objects are locked in Visier. Changes start in the SKL. *This requires coordinated design-time and release management with SKL & Visier*
Traceable and auditable	Every published object carries its SKL ID and version, and every publish is recorded.
Vendor-neutral and reversible	A different analytics tool means a new adapter and a republish, not a re-curation.

Benefits

- **One meaning (definition, properties) everywhere:** a term, hierarchy or calculation means the same thing in Visier, Copilot, Compass and partner APIs.
- **SKL maintains a richer set of semantics than Visier can hold:** poly-hierarchy, typed relationships, mappings and deontic norms stay intact in the SKL.
- **Lower cost to serve:** one governed change replaces parallel edits in each tool, and fewer reconciliation tickets.
- **Audit trail:** every recommendation and metric traces to a versioned SKL definition, which

supports MTBH in AI and defensibility in BI & AI.

- **Reduced risk through aligned change:** versioned, reviewed & tested from SSOT and published to downstream consumption endpoints, with drift detected & prevented instead of discovered.
- **Customer-specific terminology:** customer vocabularies (alternate labels tied to personas and domains) map to a canonical set once and publish per tenant.
- **Less lock-in:** Visier is replaceable or supplementable without redoing the semantics.

What is authored versus what is published

The SKL authors everything.
Only the parts that fit a Visier object type are published; the rest stays in the SKL.

Element	Authored in the SKL as	Published into Visier as	Notes
Vocabularies (terms, definitions, synonyms)	Concepts with ID, labels, definition, status	Dimension members or property values, each with its SKL ID	Synonyms and definitions stay in the SKL
Taxonomies (hierarchies)	Concept schemes with broader/narrower relations	Leveled or parent-child dimensions (one parent path per member)	Multiple parents handled by the rule in the next section
Ontology (classes, relations, axioms)	Classes, typed relations, constraints	Not published; flattened projections only	Stays in the SKL
Mappings and crosswalks	Match relations between vocabularies (exact, close, broad, narrow)	Mapping tables used at data load	Authoritative copy stays in the SKL
Selection criteria	Declarative rule definitions	Selection concepts and calculated properties, where expressible	What Visier cannot express stays in services (1)
Rules, constraints, deontic norms	Governed logic with test cases	Not published; executed in deterministic services, with results loadable	Stays in the SKL
Calculations and metrics	Definition with test cases	Metric formula as an implementation of the SKL definition (2)	Parity-tested; the SKL definition wins
Skills	ESCO-aligned reference set plus customer terms	To be decided (3)	See the skills section
Provenance	SKL ID, version, publish record	A property on every published object	Enables drift detection

1. Those rules remain authored in the SKL and are executed by the deterministic services (the “deterministic core” in your deck). Visier doesn’t get a watered-down copy of the rule. It can still receive the *results*, for example, a flag or a classification loaded as a property, and analyze them like any other data.

   The reasons for doing it this way:
   - **No silent divergence:** an approximated rule rebuilt in Visier could give different answers from the real one.
   - **Single logic:** the rule runs in one place, so there’s one thing to test and audit.
   - **Honest limits:** Visier stays a consumer, and the SKL remains the authority.

   If a rule can’t be published, the validation step should say so explicitly, so nobody assumes Visier is enforcing something it isn’t.

2. Metric Definitions, Symbolic in the SKL and Executable in the Target

   The SKL holds each metric as a symbolic, tool-neutral expression over SKL concepts. Each target (Visier first) holds an executable implementation of it, which is generated or written from the definition and tested against it. Companion to Document.

Layer	What it holds	Who owns it
Definition (SKL)	Symbolic expression over concept IDs, period handling, aggregation, filters, units, test cases, version	Domain steward, with the semantic architect
Implementation (target)	Executable formula in the target's own syntax and objects	The target's adapter
Evidence	Parity-test results for each release	Publishing service

   Rule: If the definition and an implementation disagree, the definition is correct, and the implementation is the defect.

   Example (illustrative)

   $$\text{VoluntaryTurnoverRate}_p = \frac{\left|\{\, s \in \text{Separation}_p \mid \text{reason}(s) \in \text{Voluntary} \,\}\right|}{\overline{\text{Headcount}}_p}$$

   Separation, Voluntary, and Headcount are SKL concepts with governed definitions, and p is the period. The Visier adapter turns this into a Visier metric formula over its own subjects, dimensions, and selection concepts, whereas another target may get a different translation of the same definition.

3. Proposed content for the skills row of the scope table. There are 2 “layers” of skills that the SKL would hold and manage side by side.

   - **ESCO-aligned reference set.** European Commission’s multilingual classification of

skills, competences and occupations. The SKL carries a canonical (authoritative) set of skills built on the ESCO set, with stable IDs, labels, definitions and hierarchy. “Aligned” means the SKL set tracks ESCO’s structure and identifiers instead of inventing its own. This is critical to ensure a stable foundation for Visier’s own skills ontology. This gives us a common starting point for mapping.

- **Customer terms.** Each customer describes skills in their own words (definitions), and often has their own list of terms (alternate labels): “Payroll processing,” “payroll admin”, and “running payroll” might all be the same thing to a customer, and a customer might have a skill that ESCO doesn’t list at all. The SKL would keep those as the customer’s local terms, scoped to that customer.

Mapping Rules in SKL - Each customer term is linked to a canonical skill with a match relation:

- **Exact or close match:** the customer’s term is the same as, or nearly the same as, a canonical skill.
- **Broader or narrower:** it’s a more general or more specific version of one.
- **No match:** a governed request creates a new local or canonical skill, which a steward approves.

Benefits of this modeling approach:

- **Comparability:** Every customer term points to a shared canonical skill; we can benchmark and aggregate across customers without erasing how each one talks.
- **Respect for local language:** Fidelity of PoV so customers still see and use their own terminology.
- **Publishing:** Relevancy and consistency where results can be published per tenant to Visier (or other targets) with both the canonical ID and the customer’s label.

Note: There are open questions about the mapping of Visier’s skill IDs to ESCO’s, whether we load alongside or replace Visier’s skills dimension, and who owns ESCO version updates and licensing.

Modeling Guidelines: Terms

Additional Properties for every term

- Output name and SKL ID
- Inputs, as SKL concept IDs
- Period and time handling
- Aggregation function
- Filters, expressed as concept membership
- Edge cases (nulls, zero denominators, partial periods)
- Units
- Test cases: given these inputs, the result must be this number
- Version

Authoring Options:

	Machine-readable expression	**Prose plus test cases**
What it is	A structured formula over concept IDs	A written definition and sample inputs and outputs
Adapter work	Generates or compiles the target formula	A person writes the target formula by hand
Drift risk	Lower; parity tests confirm the translation	Higher; tests are the only proof of a match
Recommendation	Use for pilot metrics	Acceptable only as a stopgap

Modeling Rules:

1. The SKL definition wins, and changes are made only in the SKL (SSOT)
2. The adapter translates the definition. Where the target cannot express part of it, the adapter reports it as untranslatable and does not approximate it (key for analytic governance)
3. Parity tests run on every release and block the publish on failure (QBD)
4. If a metric cannot be expressed by the target (Visier endpoint), then it should be computed in the semantic outbound services so the result can be loaded into the target as data

Modeling Guidelines: Skills

Recommendation: Host the ESCO-aligned skills ontology in the SKL as the canonical reference, treat each customer's own skills and terminology as local terms mapped to it, and publish the result downstream per tenant. This status is TBD pending the open questions below.

Options	What it means	Trade-off
A. Map ESCO in the SKL, expose downstream (proposed)	The SKL holds the canonical skills and customer terms, linked by match relations (exact, close, broad, narrow). Publishing writes the mapped result into Visier.	Control & reuse - requires reconciling IDs with Visier's own skills ontology.
B. Adopt Visier's skills ontology as is	Use the shipped 14,000+ skills and their hierarchy.	Speed - customer terms and customization are limited, and customization (future option)
C. Per-customer skill sets only	Each customer's skills load as their own dimension.	Fits each customer - no shared canonical set, so benchmarking and reuse suffer.

A few questions:

- How do Visier's skill IDs relate to ESCO identifiers, and how much has Visier changed from ESCO?
- Do we load alongside Visier's skills dimension or replace it, and what does that do to its prebuilt metrics?
- Which ESCO version and licence terms apply, and who owns updates?
- How do we handle version control in SKL (a) and publishing to endpoints (b)?
- Who approves changes (terms, mappings) for customer labels?
- How do we handle & approve customer terms that do not map to canonical terms?

Operational Recommendations:

1. **Consumption only downstream.** Published dimensions, concepts, and metrics in Visier are locked. Any change starts in the SKL and flows through a publish. A reconciliation job flags any object in Visier that differs from its SKL source.

2. **What stays in the SKL.** Ontology axioms and relations, mappings, rules, constraints, deontic norms, MTBH logic, and eligibility and compliance calculations. Services execute the logic. Visier receives results and dimensions.

3. **Concepts with multiple parents use a polymorphic relation.** The canonical concept stays single in the SKL. For each additional parent context, the SKL mints a new fully qualified term (for example, Parent A > Term) with its own ID, a reference to that parent, and a link to the canonical concept ID. Visier receives one parent path per term, and the canonical ID is carried as a property so analysis can still roll up by the single concept. Trade-off: summing across contexts double-counts, so metrics must count by canonical ID or use one chosen view.

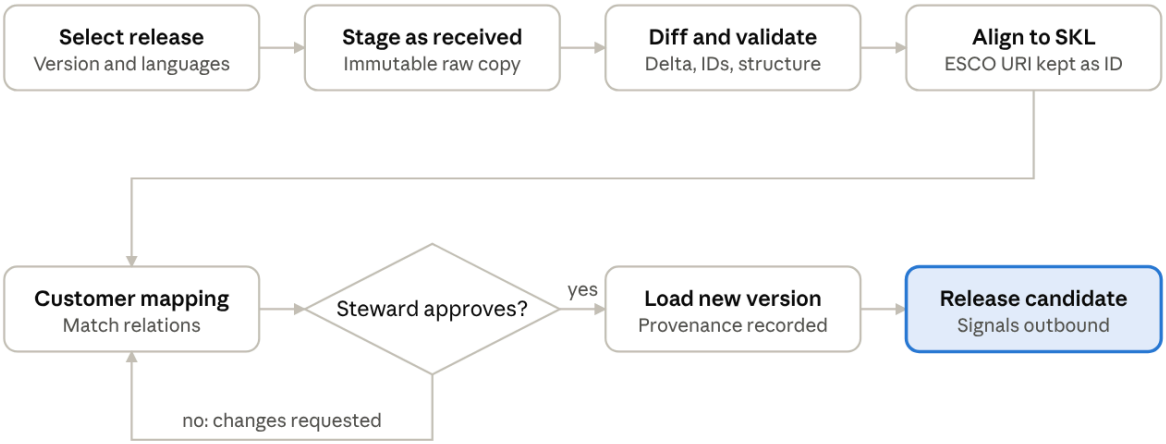
4. **Publishing goes through the API layer.** The publishing service calls Visier's APIs, batching changes into a project (using the ProjectID header) so a publish lands as one production version instead of hundreds. Fallback: file-based or Direct Data Intake loading. Gate: confirm the status of the Dimensions API (currently Alpha) and its dimension-type limits with Visier before committing to it.

5. **Versioned, reviewed releases.** Each publish has a version, release notes, an approval, and a rollback path. Nothing reaches production Visier without passing validation and the parity checks.

Semantic Inbound Services (ingestion ESCO models into SKL)

ESCO is published as downloads (SKOS-RDF, CSV, and other formats) and through services and local APIs, and it ships delta files between versions. The inbound service loads one release at a time.

Nothing from ESCO reaches the SKL without validation and steward approval

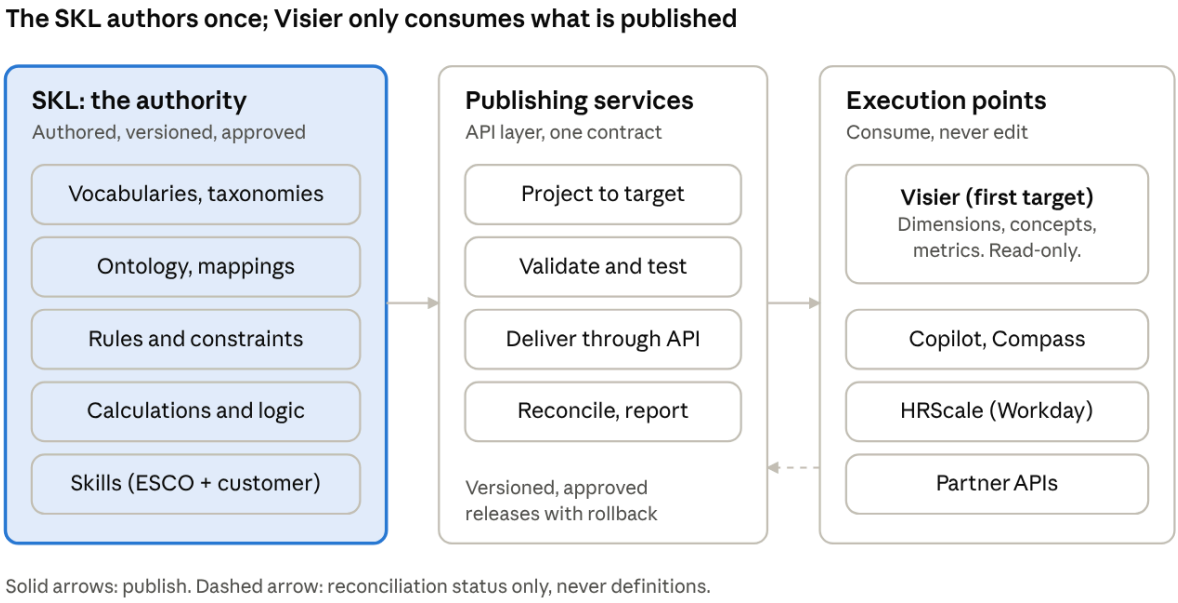


Source: ESCO downloads (SKOS-RDF, CSV and others) or its APIs.
Delta files list what changed between ESCO versions.

Note: A rejected release goes back to customer mapping with the steward's requested changes. Only an approved release becomes a new SKL version and a candidate for publishing.

Semantic Outbound Services (publishing)

The SOS (publishing) is a service with one contract and one adapter per execution point, so adding a target does not change the SKL (isolation, scalability, resiliency). Below is the one-way flow (pub-sub):



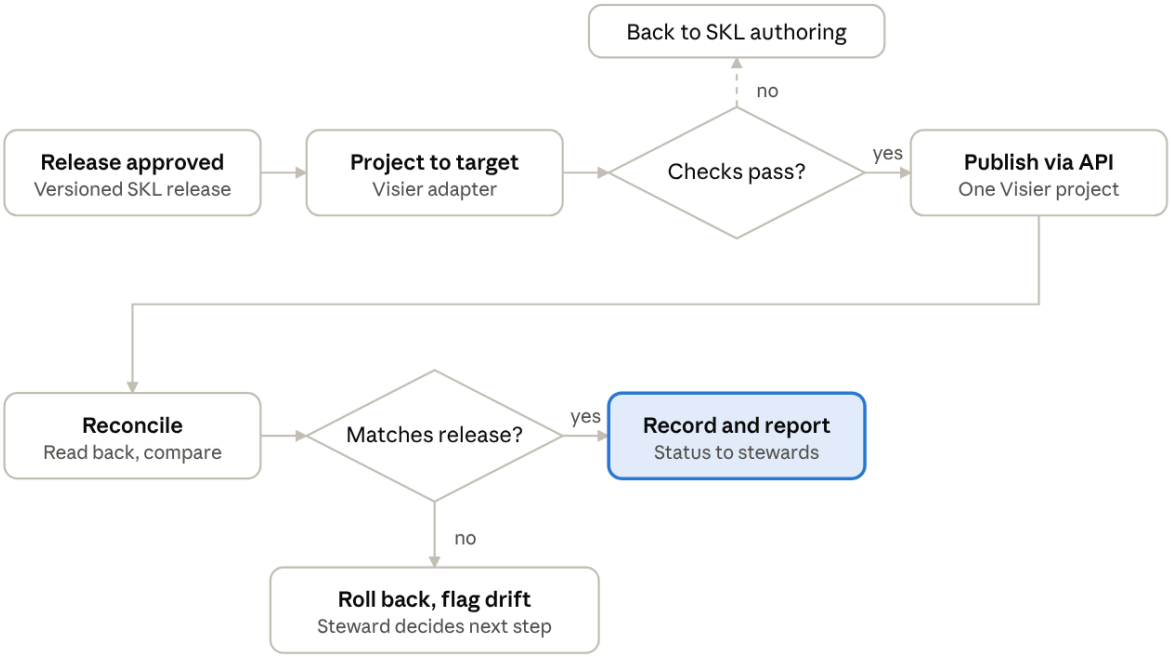
Semantic Outbound Services (publishing into Visier)

The outbound service takes only an approved release and runs the same steps for every target, with a Visier adapter doing the translation.

Publishing Steps:

- **Publish:** projects a versioned SKL release into the target's shape and delivers it.
- **Validate:** checks structure (for example, the dimension types the target supports), unique IDs, and parity tests before anything is delivered.
- **Reconcile:** reads back what the target holds and compares it to the release.
- **Status and rollback:** reports release state and reverts to the previous version.
- **Visier adapter:** maps concept schemes to Visier dimension types and selection concepts, writes through Visier's APIs inside a project, and stamps the SKL ID and version on every published object.

A release reaches Visier only after checks pass, and is reconciled afterwards



Anything the target cannot express is reported as untranslatable, not approximated.

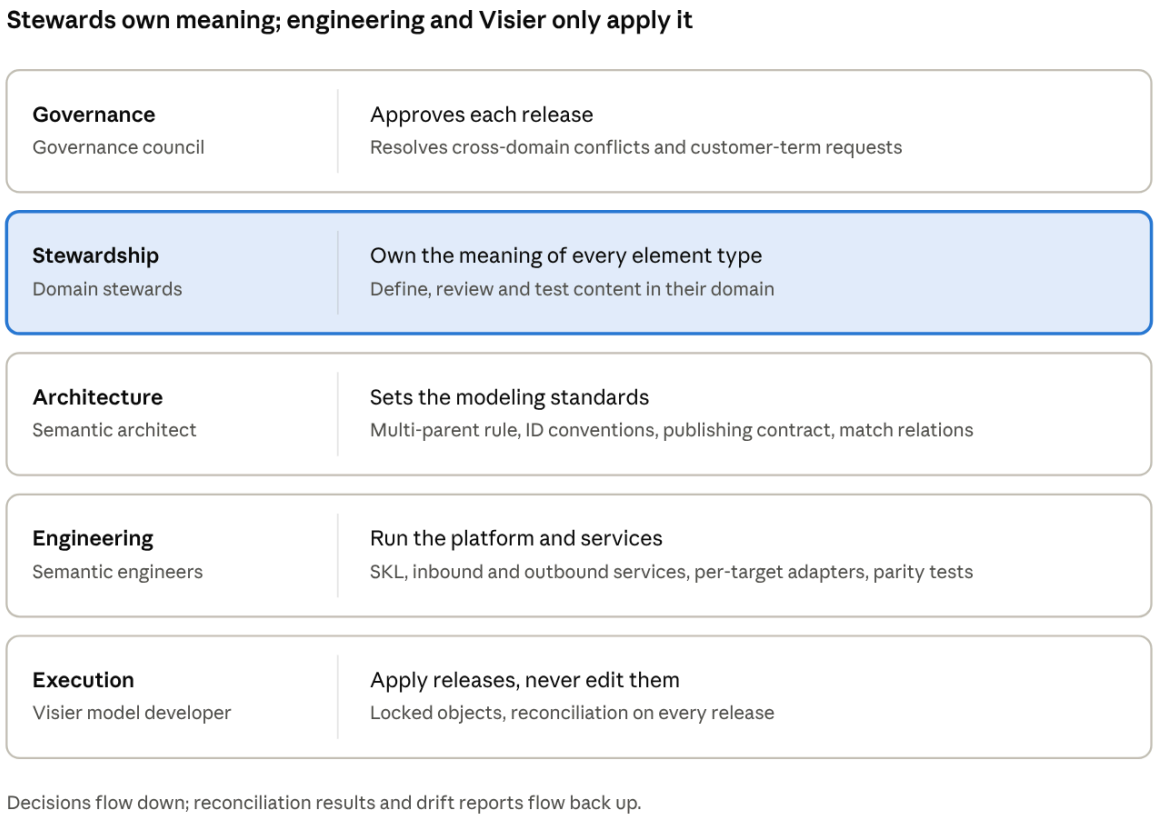
Failed checks send the work back to SKL authoring and never reach Visier. After a publish, the service reads Visier back and compares it with the release; a mismatch triggers rollback and a drift flag for the steward.

Proposed Target Operating Model (TOM):

The model follows from seven first principles. Everything else (roles, process, measures) is derived from them.

1. **Meaning has one owner.** Every term, rule, and calculation has one accountable steward and one home, the SKL.
2. **Define once, publish many.** Targets are projections, never sources.
3. **Separate definition from execution.** The SKL defines; services and platforms execute.
4. **Downstream consumes, never edits.** Anything changed downstream is drift, and drift is a defect.
5. **Every published fact is traceable.** It carries its SKL ID, version, and publish record.
6. **Automate the mechanical, govern the judgemental.** Projection, validation, and reconciliation are automated. Naming, scoping, and approving meaning are human decisions.
7. **Measure the system.** If we don't measure drift, lead time, and cost to serve, the model will not hold.

Five layers, each with a named role and a clear decision right. Meaning has exactly one owner, the domain steward; everything below that layer applies it.



The layer boundaries follow the first principles in the main proposal: define once, separate

definition from execution, and let downstream consume without editing.

Operational Responsibilities

Role	Accountable for
Semantic architect	SKL structure, modeling standards, the publishing contract, and the multi-parent rule
Domain stewards (HR, benefits, compliance)	Meaning and correctness of terms, rules, and calculations in their domain
Semantic engineers	The SKL platform, publishing services and per-target adapters
Visier model developer	Applying published releases in Visier, and blocking local edits to published objects
Governance council	Approving releases, resolving cross-domain conflicts and customer-term requests

Element types: modeling and ownership

Each of the six element types has its own source, steward, and check. Rows marked proposed are design choices not yet agreed.

Element	Modeled in the SKL as	Source	Steward	Published to Visier as	Quality check
Terms	Concept with ID, label, definition, synonyms, & status	Authored by stewards	Domain steward	Dimension members or property values	Definition review; duplicate and synonym check
Roles (proposed)	Concept linked to occupations by match relations	Customer and authored	HR steward	Role dimension members	Every role mapped or flagged
Skills	ESCO-aligned concept scheme plus customer terms linked by match relations	ESCO inbound; customer terms	Skills steward	Skills dimension (to be decided)	ID alignment to ESCO; queue of unmapped terms
Occupations	ESCO-aligned concept scheme with occupation-to-skill relations	ESCO inbound	Skills steward	Occupation dimension, relations flattened	Release diff against ESCO delta file
Rules	Governed logic	Authored by	Compliance	Selection	Test cases pass;

Element	Modeled in the SKL as	Source	Steward	Published to Visier as	Quality check
	with test cases	stewards	or benefits steward	concepts where expressible; otherwise run in services	untranslatable report
Measures	Symbolic definition over concept IDs with test cases	Authored by stewards	Analytics steward, with the semantic architect	Metric formula as an implementation	Parity test on every release

Lifecycle Flow:

1. **Propose:** a steward raises a change with its business reason.
2. **Author:** the change is modeled in the SKL with definitions and test cases.
3. **Review:** the steward and architect check meaning and fit.
4. **Validate:** automated checks run on structure, IDs, and parity tests.
5. **Approve and release:** the council approves a versioned release.
6. **Publish:** the service delivers it to Visier through the API layer, inside a project.
7. **Reconcile:** the service reads back Visier and flags any difference from the release.
8. **Learn:** drift and incidents feed back into standards and tests.

Measures:

- Share of Visier semantic objects sourced from the SKL
- Drift incidents per release, and time to resolve
- Parity-test failures caught before publish
- Lead time from approved change to live in Visier
- Cost to serve for the downturn advisory pilot, as in the baseline the deck proposes

Risks, pilot and next steps

Pilot scope: the downturn workforce advisory vocabularies (for example, employment actions such as furlough, reduced hours, and reduction in force, with their reasons and benefit classes). Author them in the SKL, publish a small set of dimensions and selection concepts into Visier, and run the reconciliation and parity checks end to end—baseline cost to serve first.

Risk	Mitigation
Dimensions API is Alpha and limited to leveled and parent-child types	Confirm status and roadmap with Visier; keep file-based or Direct Data Intake loading as the fallback
Visier admins edit published objects	Lock them, and reconcile on every release
Flattening loses semantics (multiple parents, relations)	Fully qualified terms with canonical IDs; keep ontology in the SKL
Summing across qualified terms double-counts	Metrics count by canonical ID, or use a single chosen view
Calculation drift between the SKL and Visier metrics	Test cases in the SKL, with parity tests on every release
Skills IDs do not line up with ESCO or Visier's ontology	Resolve in the skills decision before any skills publish